

Especificação Técnica - Sistema de Atendimento Gratia Localiza

Este documento detalha a previsão técnica para a implementação do sistema, seguindo o padrão de caminhos, ações e pseudocódigos solicitados.

1. Modelagem de Dados e Configuração

`/backend/app/models.py`

- **canção** : criar
- **descrição** : Definição das entidades do banco de dados utilizando SQLAlchemy. O sistema deve oferecer suporte aos usuários com diferentes papéis e tickets de atendimento com histórico de respostas.
- **pseudocódigo** :

```
CLASSE User(UserMixin, db.Model):
    ID = Coluna(Inteiro, Chave Primária)
    NOME = Coluna(String(100), Obrigatório)
    EMAIL = Coluna(String(100), Único, Obrigatório)
    SENHA_HASH = Coluna(String(200), Obrigatório)
    PAPEL = Coluna(String(20), Padrao="cliente") # admin, atendente, cliente
    TICKETS = Relacionamento("Ticket", backref="autor")
```

```
CLASSE Ticket(db.Model):
    ID = Coluna(Inteiro, Chave Primária)
    TITULO = Coluna(String(100), Obrigatório)
    DESCRICAO = Coluna(Texto, Obrigatório)
    STATUS = Coluna(String(20), Padrao="aberto") # aberto, respondido, fechado
    DATA_CRIACAO = Coluna(DateTime, Padrao=agora)
    USER_ID = Coluna(Inteiro, Chave Estrangeira(user.id))
    RESPOSTAS = Relacionamento("Response", backref="ticket")
```

```
CLASSE Response(db.Model):
    ID = Coluna(Inteiro, Chave Primária)
    CONTEUDO = Coluna(Texto, Obrigatório)
    DATA_CRIACAO = Coluna(DateTime, Padrao=agora)
```

```
TICKET_ID = Coluna(Inteiro, Chave Estrangeira(ticket.id))
USER_ID = Coluna(Inteiro, Chave Estrangeira(user.id))
```

/backend/app/__init__.py

- **canção** : criar
- **descrição** : Inicialização da aplicação Flask (Application Factory), configuração das extensões e registro dos Blueprints.
- **pseudocódigo** :

```
FUNCAO create_app():
    APP = Flask(__name__)
    APP.CONFIG.from_object(Config)

    DB.init_app(APP)
    LOGIN_MANAGER.init_app(APP)
    MIGRATE.init_app(APP, DB)

    REGISTRAR_BLUEPRINT(auth_bp)
    REGISTRAR_BLUEPRINT(customer_bp)
    REGISTRAR_BLUEPRINT(attendant_bp)
    REGISTRAR_BLUEPRINT(admin_bp)

    RETORNAR APP
```

2. Autenticação e Gestão de Acesso

/backend/app/routes/auth.py

- **canção** : criar
- **descrição** : Gerenciamento de login, logout e autocadastro de clientes.
- **pseudocódigo** :

```
ROTA "/login" [GET, POST]:
    SE método == POST:
        USUARIO = BuscarUserPorEmail(request.form.email)
        SE USUARIO existe E verificar_senha(USUARIO, request.form.password) == True:
            login_user(USUARIO)
            REDIRECIONAR para dashboard conforme USUARIO.PAPEL
        SENAO:
            ExibirErro("Credenciais inválidas")
```

```
RETORNAR render_template("login.html")
```

```
ROTA "/register" [GET, POST]:
```

```
SE método == POST:
```

```
    NOVO_USER = User(nome, email, papel="cliente")
```

```
    definir_senha(NOVO_USER, request.form.password)
```

```
    SalvarNoDB(NOVO_USER)
```

```
    REDIRECIONAR para login
```

```
RETORNAR render_template("register.html")
```

3. Módulo do Cliente

[/backend/app/routes/customer.py](#)

- **canção** : criar
- **descrição** : Interface para o cliente enviar e consultar suas interferências.
- **pseudocódigo** :

```
ROTA "/customer/tickets" [GET]:
```

```
    LISTA_TICKETS = BuscarTickets(user_id=current_user.id)
```

```
    RETORNAR render_template("customer/list.html", LISTA_TICKETS)
```

```
ROTA "/customer/ticket/new" [GET, POST]:
```

```
SE método == POST:
```

```
    TICKET = Ticket(titulo, descricao, user_id=current_user.id)
```

```
    SalvarNoDB(TICKET)
```

```
    REDIRECIONAR para "/customer/tickets"
```

```
RETORNAR render_template("customer/new.html")
```

4. êxtase do Entendente

[/backend/app/routes/attendant.py](#)

- **canção** : criar
- **descrição** : Funcionalidades para visualizar tickets pendentes e responder propostas.
- **pseudocódigo** :

```
ROTA "/attendant/tickets" [GET]:
```

```
    TICKETS_PENDENTES = BuscarTickets(status="aberto")
```

```
    RETORNAR render_template("attendant/dashboard.html", TICKETS_PENDENTE
```

```
ROTA "/attendant/ticket/<id>/reply" [POST]:
    RESPOSTA = Response(conteudo, ticket_id=id, user_id=current_user.id)
    TICKET = BuscarTicketPorId(id)
    TICKET.status = "respondido"
    SalvarNoDB(RESPOSTA, TICKET)
    REDIRECIONAR para "/attendant/tickets"
```

5. Módulo do Administrador

/backend/app/routes/admin.py

- **canção** : criar
- **descrição** : CRUD de atendentes e geração de relatórios gerenciais básicos.
- **pseudocódigo** :

```
ROTA "/admin/attendants" [GET, POST]:
    SE método == POST: # Criar atendente
        NOVO_ATENDENTE = User(nome, email, papel="atendente")
        definir_senha(NOVO_ATENDENTE, password)
        SalvarNoDB(NOVO_ATENDENTE)
    LISTA_ATENDENTES = BuscarUsuariosPorPapel("atendente")
    RETORNAR render_template("admin/attendants.html", LISTA_ATENDENTES)

ROTA "/admin/reports" [GET]:
    TOTAL_TICKETS = ContarTodosTickets()
    TICKETS_POR_STATUS = AgruparTicketsPorStatus()
    RETORNAR render_template("admin/reports.html", TOTAL_TICKETS, TICKETS)
```

6. Utilitários

/backend/app/utils/decorators.py

- **canção** : criar
- **descrição** : Decoradores para restrições de acesso às rotas baseadas no papel do usuário.
- **pseudocódigo** :

```
FUNCAO role_required(papel_esperado):
    DECORADOR(f):
        SE current_user.papel != papel_esperado:
```

```
Abort(403) # Acesso negado  
RETORNAR f()
```